

kafka 핵심 용어 개념

카프카 핵심 용어 가이드

1. 4대 핵심 구성 요소

용어	비유 (우체국)	설명
브로커(Broker)	우체국 지점	카프카 서버 그 자체입니다. 데이터를 저장하고 관리하는 물리적 서버입니다.
토픽(Topic)	우편함(주소)	데이터가 쌓이는 방입니다. '결제', '로그'처럼 이름표를 붙여 구분합니다.
프로듀서(Producer)	편지 쓰는 사람	데이터를 생성해서 카프카 토픽으로 보내는 앱입니다. kafka 커넥터 사용시 별도의 코딩 없이 프로듀싱 기능 가능(oracle의 redo 로그를 읽어오는 devezium)
컨슈머(Consumer)	편지 읽는 사람	토픽에서 데이터를 가져와 처리하는 앱(Flink, Spark 등)입니다.

2. 내부 동작 필수 개념

용어	설명	비유 및 특징
파티션(Partition)	토픽을 쪼갠 단위	우편함을 칸칸이 나눠서 병렬 처리 가 가능하게 함
오프셋(Offset)	메시지의 고유 번호	"어디까지 읽었나" 표시하는 책갈피 역할 (데이터 유실 방지)
레플리카(Replica)	데이터 복제본	서버 한 대가 죽어도 데이터가 안 사라지게 하는 안전장치

3. 데이터 흐름 한 눈에 보기

1. **Producer**: "로그 데이터(Topic)를 Broker에 넣어줘!"
2. **Broker**: "알았어, 7번 Partition에 Offset 100번으로 저장할게."
3. **Consumer**: "나 Offset 99번까지 읽었으니까 다음 데이터(100번) 가져갈게!"

4. 그럼 여기서 kafka-Connect란?

카프카 커넥트 (Kafka Connect)

- **정체:** 프레임워크 (시스템 그 자체)
- **설명:** 커넥터들이 안정적으로 실행될 수 있도록 환경을 제공하는 **플랫폼**입니다.
- **비유:** 여러 개의 게임(커넥터)을 돌리기 위한 **게임기 본체**나 **운영체제(OS)**와 같습니다.
- **대화 예시:** "우리 서버에 **카프카 커넥트** 설치됐나요?"

2. 카프카 커넥터 (Kafka Connector)

- **정체:** 플러그인 (개별 부품)
- **설명:** 실제로 특정 시스템(Oracle, MinIO 등)에 접속해서 데이터를 옮기는 **실행 단위**입니다.
- **비유:** 게임기 본체에 꽂아서 사용하는 개별 **게임 팩**이나 **앱(App)**과 같습니다.
- **대화 예시:** "오라클용 **디bezium 커넥터** 설정 좀 봐주세요."

카프카 커넥터(Kafka Connector) 핵심 정리

1. 🔍 한 줄 정의: "데이터 배달용 전용 어댑터"

카프카 커넥터는 직접 코딩을 하지 않고도, **설정 파일(JSON)**만 작성하면 특정 시스템(DB, 파일, 클라우드 저장소 등)과 카프카를 자동으로 연결해 주는 도구입니다.

💡 아주 쉬운 비유

- **프로듀서/컨슈머:** 내가 직접 재료를 사다가 요리하는 '**직접 요리**' (코딩 필요)
- **커넥터:** 전자레인지에 돌리기만 하면 되는 '**밀키트**' (설정만 필요)

2. ↑ 두 가지 방향: 소스(Source)와 싱크(Sink)

커넥터는 데이터가 흐르는 방향에 따라 딱 두 종류로 나뉩니다.

종류	역할	비유	데이터 흐름
소스 커넥터 (Source)	외부 데이터를 카프카로 가져 오기	입력(Input) 빨대	외부 시스템 → Kafka
싱크 커넥터 (Sink)	카프카 데이터를 외부로 내보 내기	출력(Output) 수도꼭 지	Kafka → 외부 시스 템

Apache Kafka의 핵심 동작 원리

카프카를 지탱하는 3가지 설계 철학은 **로그(Log)**, **분산(Distributed)**, **순차적 처리(Sequential)**입니다.

1. 로그 기반의 저장 방식 (Append-only Log)

카프카는 데이터를 데이터베이스처럼 수정하거나 삭제하지 않습니다. 들어오는 데이터를 파일 끝에 단순히 이어 붙이는(Append) **로그 파일** 형태를 유지합니다.

- **원리:** 파일 끝에 붙이기만 하면 되기 때문에 디스크 쓰기 속도가 메모리만큼 빠릅니다 (Sequential I/O).
- **결과:** 데이터가 쌓여 있어도 성능 저하가 거의 없습니다.

2. 토픽(Topic)과 파티션(Partition)

카프카는 데이터를 **토픽**이라는 주제로 나누고, 이를 다시 여러 개의 **파티션**으로 쪼개서 분산 저장합니다.

- **원리:** 한 대의 서버가 감당하기 힘든 대량의 데이터를 여러 서버(Broker)에 나누어 담습니다.
- **병렬 처리:** 파티션이 여러 개이므로, 컨슈머(Consumer) 여러 대가 동시에 데이터를 읽어갈 수 있어 처리량이 기하급수적으로 늘어납니다.

3. 오프셋(Offset) 관리

파티션 내부에서 각 메시지는 고유한 번호인 **오프셋(Offset)**을 가집니다.

- **원리:** 컨슈머는 "내가 몇 번 오프셋까지 읽었어"라는 정보만 기록합니다.
- **장점:** 컨슈머가 죽었다 살아나도 기록된 오프셋 다음부터 읽으면 되므로 데이터 유실이나 중복 처리를 방지할 수 있습니다.

4. 복제와 고가용성 (Replication)

카프카는 파티션을 여러 대의 브로커에 복제해둡니다.

- **원리:** 하나는 **리더(Leader)**, 나머지는 ****팔로워(Follower)****가 됩니다.
- **고가용성:** 리더 브로커가 고장 나도 팔로워 중 하나가 즉시 리더가 되어 서비스를 중단 없이 이어갑니다.